

GUÍA PROFESIONAL DE ARREGLOS EN C

Arreglos enteros y flotantes: llenado, impresión, operaciones y análisis de resultados

Nivel sugerido	Introducción a la programación
Lenguaje	C
Tema	Arreglos unidimensionales
Objetivo	Resolver ejercicios básicos con arreglos de tipo int y float

Propósito de la guía. Que el estudiante identifique cómo declarar, llenar, imprimir y procesar arreglos, observando paso a paso cómo cambia el contenido de cada posición.

1. Concepto básico de arreglo

Un arreglo es una estructura que permite guardar varios datos del mismo tipo usando un solo nombre. Cada dato se almacena en una posición llamada índice. En C, el primer índice siempre es 0.

```
int numeros[5];  
float calificaciones[5];
```

Representación inicial de int numeros[5]:

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4
?	?	?	?	?

Aunque el arreglo tiene 5 elementos, sus índices van de 0 a 4. Este detalle es clave para evitar errores al usar ciclos.

2. Método general para resolver ejercicios con arreglos

Paso	Acción	Descripción
1	Declarar	Crear el arreglo con su tipo y tamaño.
2	Llenar	Capturar los datos usando scanf dentro de un ciclo for.
3	Imprimir	Mostrar el contenido usando printf dentro de otro ciclo for.
4	Procesar	Aplicar la operación solicitada: cuadrado, suma, promedio, mayor, menor, etc.
5	Mostrar resultado	Imprimir el resultado final de manera clara.

```
for(int i = 0; i < n; i++)
{
    // Aquí se trabaja con arreglo[i]
}
```

3. Ejercicio guiado 1: Arreglo entero y cuadrado de cada elemento

Problema: Leer 5 números enteros, guardarlos en un arreglo, imprimir el arreglo original y después imprimir el cuadrado de cada elemento.

3.1 Análisis del problema

Tipo de arreglo: int. Tamaño: 5 elementos. Operación principal: elevar cada número al cuadrado multiplicándolo por sí mismo.

Ejemplo de arreglo después de llenarlo:

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4
2	4	6	8	10

Arreglo resultante al calcular el cuadrado:

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4
4	16	36	64	100

3.2 Código completo en C

```
#include <stdio.h>

int main()
{
    int numeros[5];
    int cuadrados[5];
    int i;

    printf("Llenado del arreglo de enteros\n");
```

```

for(i = 0; i < 5; i++)
{
    printf("Ingrese el numero de la posicion %d: ", i);
    scanf("%d", &numeros[i]);
}

printf("\nArreglo original:\n");
for(i = 0; i < 5; i++)
{
    printf("numeros[%d] = %d\n", i, numeros[i]);
}

for(i = 0; i < 5; i++)
{
    cuadrados[i] = numeros[i] * numeros[i];
}

printf("\nArreglo con cuadrados:\n");
for(i = 0; i < 5; i++)
{
    printf("cuadrados[%d] = %d\n", i, cuadrados[i]);
}

return 0;
}

```

3.3 Ejecución de ejemplo

Llenado del arreglo de enteros

Ingrese el numero de la posicion 0: 2

Ingrese el numero de la posicion 1: 4

Ingrese el numero de la posicion 2: 6

Ingrese el numero de la posicion 3: 8

Ingrese el numero de la posicion 4: 10

Arreglo original:

```
numeros[0] = 2  
numeros[1] = 4  
numeros[2] = 6  
numeros[3] = 8  
numeros[4] = 10
```

Arreglo con cuadrados:

```
cuadrados[0] = 4  
cuadrados[1] = 16  
cuadrados[2] = 36  
cuadrados[3] = 64  
cuadrados[4] = 100
```

3.4 Explicación paso a paso

- Se declara `numeros[5]` para guardar los datos originales.
- Se declara `cuadrados[5]` para guardar el resultado de cada operación.
- El primer ciclo for llena el arreglo posicion por posicion.
- El segundo ciclo imprime el contenido original.
- El tercer ciclo calcula `cuadrados[i] = numeros[i] * numeros[i]`.
- El último ciclo imprime el arreglo de resultados.

4. Ejercicio guiado 2: Arreglo flotante, promedio y aumento porcentual

Problema: Leer 5 calificaciones con decimales, guardarlas en un arreglo float, imprimirlas, calcular el promedio y después generar un segundo arreglo con una bonificación del 10%.

4.1 Análisis del problema

Arreglo de calificaciones capturado:

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4
8.5	7.0	9.5	10.0	8.0

Arreglo con bonificación del 10%:

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4
9.35	7.70	10.45	11.00	8.80

Nota didáctica: en un caso real se podría limitar la calificación máxima a 10. En este ejercicio no se limita para observar claramente la operación matemática.

4.2 Código completo en C

```
#include <stdio.h>

int main()
{
    float calificaciones[5];
    float bono[5];
    float suma = 0;
    float promedio;
    int i;

    printf("Llenado del arreglo de calificaciones\n");

    for(i = 0; i < 5; i++)
    {
        printf("Ingrese la calificacion de la posicion %d: ", i);
        scanf("%f", &calificaciones[i]);
    }

    printf("\nArreglo original:\n");
    for(i = 0; i < 5; i++)
    {
        printf("calificaciones[%d] = %.2f\n", i, calificaciones[i]);
    }

    for(i = 0; i < 5; i++)
    {
```

```

    suma = suma + calificaciones[i];
}

promedio = suma / 5;

for(i = 0; i < 5; i++)
{
    bono[i] = calificaciones[i] * 1.10;
}

printf("\nPromedio = %.2f\n", promedio);

printf("\nArreglo con bonificacion del 10 por ciento:\n");
for(i = 0; i < 5; i++)
{
    printf("bono[%d] = %.2f\n", i, bono[i]);
}

return 0;
}

```

4.3 Ejecución de ejemplo

Llenado del arreglo de calificaciones

Ingrese la calificacion de la posicion 0: 8.5

Ingrese la calificacion de la posicion 1: 7.0

Ingrese la calificacion de la posicion 2: 9.5

Ingrese la calificacion de la posicion 3: 10.0

Ingrese la calificacion de la posicion 4: 8.0

Arreglo original:

calificaciones[0] = 8.50

calificaciones[1] = 7.00

calificaciones[2] = 9.50

```
calificaciones[3] = 10.00
```

```
calificaciones[4] = 8.00
```

```
Promedio = 8.60
```

Arreglo con bonificacion del 10 por ciento:

```
bono[0] = 9.35
```

```
bono[1] = 7.70
```

```
bono[2] = 10.45
```

```
bono[3] = 11.00
```

```
bono[4] = 8.80
```

4.4 Explicación paso a paso

- calificaciones[5] guarda los datos capturados por el usuario.
- suma inicia en 0 porque ahí se acumularán los valores.
- El promedio se obtiene dividiendo suma entre 5.
- `bono[i] = calificaciones[i] * 1.10` aumenta cada dato en 10%.
- `%.2f` imprime números flotantes con dos decimales.

5. Errores comunes y cómo evitarlos

Error común	Corrección
Usar <code>i <= 5</code> en un arreglo de 5 elementos	Debe usarse <code>i < 5</code> porque los índices válidos son 0, 1, 2, 3 y 4.
Olvidar <code>&</code> en <code>scanf</code>	Para guardar en una posición del arreglo se usa <code>&arreglo[i]</code> .
No inicializar suma	Antes de acumular valores, suma debe iniciar en 0.
Confundir <code>int</code> con <code>float</code>	Use <code>%d</code> para <code>int</code> y <code>%f</code> para <code>float</code> en <code>scanf</code> . Para imprimir <code>float</code> puede usar <code>%.2f</code> .
Modificar el arreglo original sin querer	Si necesita conservar los datos originales, use un segundo arreglo para guardar los resultados.

6. Diez ejercicios propuestos para resolver

Resuelve los siguientes ejercicios usando arreglos, ciclos for y operaciones básicas. En cada ejercicio escribe: declaración, llenado, impresión, procesamiento y salida final.

1. Imprimir arreglo entero

Leer 10 números enteros e imprimirlos en el mismo orden.

- Entrada: 3 5 7 9 11
- Arreglo: [3][5][7][9][11]...
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

2. Multiplicar por 2

Leer 10 enteros y mostrar cada elemento multiplicado por 2.

- Entrada: 1 2 3 4 5
- Resultado: [2][4][6][8][10]
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

3. Calcular cuadrados

Leer 10 enteros y guardar sus cuadrados en otro arreglo.

- Entrada: 2 3 4 5 6
- Resultado: [4][9][16][25][36]
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

4. Sumar 5 unidades

Leer 10 enteros y generar otro arreglo sumando 5 a cada elemento.

- Entrada: 10 20 30
- Resultado: [15][25][35]
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

5. Encontrar el mayor

Leer 10 enteros y determinar el número más grande.

- Entrada: 8 3 15 4 9
- Mayor: 15
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

6. Encontrar el menor

Leer 10 enteros y determinar el número más pequeño.

- Entrada: 8 3 15 4 9
- Menor: 3
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

7. Promedio flotante

Leer 10 números float y calcular su promedio.

- Entrada: 8.5 7.0 9.5
- Promedio parcial: 8.33
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

8. Contar positivos y negativos

Leer 10 enteros y contar cuántos son positivos y cuántos son negativos.

- Entrada: 2 -1 5 -8 0
- Positivos: 2, Negativos: 2
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

9. Invertir arreglo

Leer 10 enteros e imprimirlos en orden inverso.

- Entrada: 1 2 3 4 5
- Salida: 5 4 3 2 1
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

10. Comparar dos arreglos

Leer dos arreglos de 5 enteros y contar coincidencias por posición.

- A: 1 2 3 4 5; B: 1 8 3 7 5
- Coincidencias: 3
- Pista: usa un ciclo for para recorrer las posiciones del arreglo.

7. Reto integrador

Crear un programa en C que lea 10 números enteros y calcule: suma, promedio, mayor, menor y cuadrado de cada elemento. Debe usarse un arreglo original y, si se desea, un segundo arreglo para guardar cuadrados.

Estructura sugerida:

1. Declarar arreglo original[10]
2. Declarar arreglo cuadrados[10]
3. Llenar original[10]
4. Imprimir original[10]
5. Calcular suma, mayor y menor
6. Calcular promedio
7. Calcular cuadrados[10]
8. Imprimir todos los resultados

8. Plantilla general para resolver cualquier ejercicio

```
#include <stdio.h>

int main()
{
    // 1. Declarar variables y arreglos

    // 2. Llenar arreglo
    for(int i = 0; i < TAM; i++)
    {
        // scanf(...)
    }

    // 3. Procesar arreglo
    for(int i = 0; i < TAM; i++)
    {
        // operacion
    }
}
```

```
}

// 4. Imprimir resultados
for(int i = 0; i < TAM; i++)
{
    // printf(...)
}

return 0;
}
```

CAPÍTULO 1. VARIABLES Y TIPOS DE DATOS

Una variable es un espacio en memoria utilizado para almacenar información.

```
int edad = 20;
float promedio = 8.5;
```

Tipos comunes:

- int
- float
- char
- double

CAPÍTULO 2. ENTRADA Y SALIDA DE DATOS

PRINTF()

printf() permite mostrar información en pantalla.

Ejemplo:

```
printf("Hola Mundo");
```

Imprimir una variable:

```
int edad=20;  
printf("%d", edad);
```

Imprimir varias variables:

```
int edad=20;  
float peso=72.5;  
printf("Edad=%d Peso=%.2f", edad, peso);
```

SCANF()

scanf() permite capturar datos desde el teclado.

```
int edad;  
scanf("%d",&edad);
```

El símbolo & indica la dirección de memoria donde se almacenará el dato.

Ejemplo completo:

```
int edad;  
float peso;
```

```
printf("Ingrese edad: ");  
scanf("%d",&edad);
```

```
printf("Ingrese peso: ");  
scanf("%f",&peso);
```

```
printf("Edad=%d Peso=%.2f",edad,peso);
```

CAPÍTULO 3. CICLOS FOR

```
for(int i=0;i<5;i++)
```

```
{  
    printf("%d",i);  
}
```

CAPÍTULO 4. ARREGLOS

Declaración:

```
int numeros[5];
```

Representación:

Índice: 0 1 2 3 4

Valor : _ _ _ _ _

Si se capturan:

5 8 3 10 7

El arreglo queda:

Índice: 0 1 2 3 4

Valor : 5 8 3 10 7

CAPÍTULO 5. EJERCICIO GUIADO 1

Obtener el cuadrado de cada elemento.

Entrada:

1 2 3 4 5

Salida:

1 4 9 16 25

CAPÍTULO 6. EJERCICIO GUIADO 2

Promedio de números flotantes.

Entrada:

8.5 7.0 9.5 10.0 8.0

Salida:

Promedio = 8.60

CAPÍTULO 7. ESTRATEGIA GENERAL

1. Declarar arreglo.
2. Llenar arreglo.
3. Recorrer arreglo.
4. Procesar información.
5. Mostrar resultados.

CAPÍTULO 8. EJERCICIOS PROPUESTOS

1. Imprimir un arreglo.
2. Multiplicar por 2.
3. Obtener cuadrados.
4. Encontrar el mayor.
5. Encontrar el menor.
6. Calcular suma.
7. Calcular promedio.
8. Contar positivos.
9. Invertir arreglo.
10. Comparar arreglos.